

Module3_USL_v4

90.80 / 100.03 points earned

26 / 28 autograded cells passed

Graded Cells

Cell 5 (cell-52f19a05e8a8b1a3)

Passed | 7.69 / 7.69 points

[View feedback](#)

Cell 6 (cell-dbe3ae17d138f001)

Passed | 15.37 / 15.37 points

[View feedback](#)

Cell 8 (cell-232d6022e1da2f62)

Try again | 0.00 / 7.69 points

[View feedback](#)

Cell 10 (cell-b1260c5ba472d43b)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 11 (cell-4a8dc462fa8d33f0)

Passed | 4.62 / 4.62 points

[View feedback](#)

Cell 12 (cell-3b1eb8e79527f012)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 13 (cell-64689afe942eba5a)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 14 (cell-a7be29cab0d2e38a)

Passed | 4.62 / 4.62 points

[View feedback](#)

Cell 15 (cell-a83193bd93c45044)

Passed | 7.69 / 7.69 points

[View feedback](#)

Cell 17 (cell-a8a3fa60da47abcf)

Passed | 7.69 / 7.69 points

[View feedback](#)

Cell 19 (cell-126fb0d12b2c66e7)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 20 (cell-9e6bc1c31e566d76)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 21 (cell-7b9c0acd25eaadda)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 22 (cell-369189aa9a403a0f)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 23 (cell-bfde47a18fccd03d)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 24 (cell-cb7766e929ce12ef)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 26 (cell-a80d917298a5c254)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 27 (cell-fdebe2b922817f71)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 28 (cell-ba2fba2da48d3160)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 29 (cell-f7e1c6f239e3155f)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 31 (cell-6103dd685a33c891)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 32 (cell-905be3dcffdd989b)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 33 (cell-53a7044088d56bae)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 34 (cell-113417d673195815)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 36 (cell-576df690bad55235)

Passed | 3.08 / 3.08 points

[View feedback](#)

Cell 37 (cell-270a003313f847ff)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 38 (cell-a5efeea7f4bf26a9)

Passed | 1.54 / 1.54 points

[View feedback](#)

Cell 39 (cell-15304b41c7ed15d3)

Try again | 0.00 / 1.54 points

[View feedback](#)

Grading

The final score that you will receive for your programming assignment is generated in relation to the total points set in your programming assignment item—not the total point value in the nbgrader notebook.

When calculating the final score shown to learners, the programming assignment takes the percentage of earned points vs. the total points provided by nbgrader and returns a score matching the equivalent percentage of the point value for the programming assignment.

DO NOT CHANGE VARIABLE OR METHOD SIGNATURES The autograder will not work properly if you change the variable or method signatures.

Validate Button

Please note that this assignment uses nbgrader to facilitate grading. You will see a **validate button** at the top of your Jupyter notebook. If you hit this button, it will run tests cases for the lab that aren't hidden. It is good to use the validate button before submitting the lab. Do know that the labs in the course contain hidden test cases. The validate button will not let you know whether these test cases pass. After submitting your lab, you can see more information about these hidden test cases in the Grader Output.

Cells with longer execution times will cause the validate button to time out and freeze. Please know that if you run into Validate time-outs, it will not affect the final submission grading.

Building Recommender Systems for Movie Rating Prediction

In this assignment, we will build a recommender systems that predict movie ratings. [MovieLense \(https://grouplens.org/datasets/movielens/\)](https://grouplens.org/datasets/movielens/) has currently 25 million user-movie ratings. Since the entire data is too big, we use a 1 million ratings subset [MovieLens 1M \(https://www.kaggle.com/odedgolden/movielens-1m-dataset\)](https://www.kaggle.com/odedgolden/movielens-1m-dataset), and we reformatted the data to make it more convenient to use.

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import time
from sklearn.model_selection import train_test_split
from scipy.sparse import coo_matrix, csr_matrix
from scipy.spatial.distance import jaccard, cosine
from pytest import approx
```

In [2]:

```
MV_users = pd.read_csv('data/users.csv')
MV_movies = pd.read_csv('data/movies.csv')
train = pd.read_csv('data/train.csv')
test = pd.read_csv('data/test.csv')
```

In [3]:

```
from collections import namedtuple
Data = namedtuple('Data', ['users', 'movies', 'train', 'test'])
data = Data(MV_users, MV_movies, train, test)
```

Starter codes

Now, we will be building a recommender system which has various techniques to predict ratings. The `class RecSys` has baseline prediction methods (such as predicting everything to 3 or to average rating of each user) and other utility functions. `class ContentBased` and `class Collaborative` inherit `class RecSys` and further add methods calculating item-item similarity matrix. You will be completing those functions using what we learned about content-based filtering and collaborative filtering.

`RecSys`'s `rating_matrix` method converts the (user id, movie id, rating) triplet from the train data (train data's ratings are known) into a utility matrix for 6040 users and 3883 movies.

Here, we create the utility matrix as a dense matrix (`numpy.array`) format for convenience. But in a real world data where hundreds of millions of users and items may exist, we won't be able to create the utility matrix in a dense matrix format (For those who are curious why, try measuring the dense matrix self.Mr using `.nbytes()`). In that case, we may use sparse matrix operations as much as possible and distributed file systems and distributed computing will be needed. Fortunately, our data is small enough to fit in a laptop/pc memory. Also, we will use `numpy` and `scipy.sparse`, which allow significantly faster calculations than calculating on `pandas.DataFrame` object.

In the `rating_matrix` method, pay attention to the index mapping as user IDs and movie IDs are not the same as array index.

In [4]:

Student's answer

(Top)

```

class RecSys():
    def __init__(self,data):
        self.data=data
        self.allusers = list(self.data.users['uID'])
        self.allmovies = list(self.data.movies['mID'])
        self.genres = list(self.data.movies.columns.drop(['mID', 'title', 'year']))
        self.mid2idx = dict(zip(self.data.movies.mID,list(range(len(self.data.movies))))))
        self.uid2idx = dict(zip(self.data.users.uID,list(range(len(self.data.users))))))
        self.Mr=self.rating_matrix()
        self.Mm=None
        self.sim=np.zeros((len(self.allmovies),len(self.allmovies)))

    def rating_matrix(self):
        """
        Convert the rating matrix to numpy array of shape (#allusers,#allmovies)
        """
        ind_movie = [self.mid2idx[x] for x in self.data.train.mID]
        ind_user = [self.uid2idx[x] for x in self.data.train.uID]
        rating_train = list(train.rating)
        return np.array(coo_matrix((rating_train, (ind_user, ind_movie)), shape=(len(self.allusers), len(self.allmovies)))).toarray()

    def predict_everything_to_3(self):
        """
        Predict everything to 3 for the test data
        """
        # your code here
        return 3*np.ones(self.data.test.rating.shape)

    def predict_to_user_average(self):
        """
        Predict to average rating for the user.
        Returns numpy array of shape (#users,)
        """
        # your code here

        def mean_rating(x):
            x = x[x!=0]
            return np.mean(x)

        avg_ratings = np.apply_along_axis(mean_rating, 1, self.Mr) #self.Mr.mean(axis=1)
        ind_user = np.array([self.uid2idx[x] for x in self.data.test.uID])
        return avg_ratings[ind_user]

        #print(self.data.test.shape, self.data.test.uID.unique(), self.data.test.mID.unique())
        #print(self.Mr.shape, self.Mr[:2,:5])
        #print(self.data.test.head())
        #print(avg_ratings.head())
        #print(self.data.test.head())
        #print(avg_ratings.shape, self.data.test.shape)
        #return np.array([avg_ratings[self.allusers.index(i)] for i in self.data.test.uID.values])

```

```

def predict_from_sim(self,uid,mid):
    """
    Predict a user rating on a movie given userID and movieID
    """
    # your code here
    ind_user = self.uid2idx[uid]
    ind_movie = self.mid2idx[mid]
    u, s = self.Mr[ind_user,:], self.sim[ind_movie,:]
    u1, s1 = u[u!=0], s[u!=0]
    return np.dot(u1, s1) / np.sum(s1) #5*

def predict(self):
    """
    Predict ratings in the test data. Returns predicted rating in a numpy a
rray of size (# of rows in testdata,)
    """
    ## your code here
    return np.apply_along_axis(lambda x: self.predict_from_sim(x[0], x[1]),
1, self.data.test.values)
    #return self.data.test.apply(lambda row : self.predict_from_sim(row['uI
D'], row['mID']), axis = 1)
    #return np.array([self.predict_from_sim(uid, mid) for (uid, mid) in zip
(self.data.test.uID, self.data.test.mID)])

def rmse(self,yp):
    yp[np.isnan(yp)]=3 #In case there is nan values in prediction, it will
impute to 3.
    yt=np.array(self.data.test.rating)
    return np.sqrt(((yt-yp)**2).mean())

class ContentBased(RecSys):
    def __init__(self,data):
        super().__init__(data)
        self.data=data
        self.Mm = self.calc_movie_feature_matrix()

    def calc_movie_feature_matrix(self):
        """
        Create movie feature matrix in a numpy array of shape (#allmovies, #gen
res)
        """
        # your code here
        return self.data.movies.drop(['mID', 'title', 'year'], axis=1)

    def calc_item_item_similarity(self):
        """
        Create item-item similarity using Jaccard similarity
        """
        # your code here
        from scipy.spatial import distance
        self.sim = 1-distance.cdist(self.Mm, self.Mm, 'jaccard') #1/(1+distanc
e.cdist(self.Mm, self.Mm, 'jaccard'))
        #print(self.sim)

class Collaborative(RecSys):
    def __init__(self,data):
        super().__init__(data)

```

```

def calc_item_item_similarity(self, simfunction, *X):
    """
    Create item-item similarity using similarity function.
    X is an optional transformed matrix of Mr
    """
    if len(X)==0:
        self.sim = simfunction()
    else:
        self.sim = simfunction(X[0]) # *X passes in a tuple format of (X,),
to X[0] will be the actual transformed matrix

def cossim(self):
    """
    Calculates item-item similarity for all pairs of items using cosine sim
ilarity (values from 0 to 1) on utility matrix
    Returns a cosine similarity matrix of size (#all movies, #all movies)
    """
    # your code here

def normalize(x):
    #x[~x.any(axis=1)] = 1
    z = np.linalg.norm(x, axis=1, keepdims=True)
    z[z == 0] = 1
    return x / z

x = self.Mr.astype(np.float)
#avg = np.apply_along_axis(lambda x: np.mean(x[x != 0], keepdims=True),
1, self.Mr)
#x -= avg
for i in range(x.shape[0]):
    x1 = x[i,:]
    avg = np.sum(x1) / np.count_nonzero(x1)
    x1[x1 == 0] = avg
    x1 = x1 - x1.mean()
    x[i,:] = x1
x = normalize(x.T)
s = x.dot(x.T)
s = (s - s.min()) / (s.max() - s.min())
#print(s.max(), s.min(), s.mean(), np.diag(s))
np.fill_diagonal(s, 1)
return s

```

```

#x = np.apply_along_axis(normalize, 1, self.Mr.T)

```

```

#x = normalize(self.Mr.T)

```

Q1. Baseline models [15 pts]

1a. Complete the function predict_everything_to_3 in the class RecSys [5 pts]

```

    """
    Calculates item-item similarity for all pairs of items using jaccard si
milarity (values from 0 to 1)
    Xr is the transformed rating matrix.
    """
    # your code here

    #pass

def pairwise_jaccard_sparse(X):
    if X.max() > 1:
        from scipy.spatial import distance

```



```

        return 1-distance.cdist(X, X, 'jaccard')
In [5]: else:
        csr = csr_matrix(X).astype(bool).astype(int)
Grade cell: cell-52f19a05e8a8b1a3 Score: 7.69 / 7.69 (Top)
        csr_rownnz = csr.getnnz(axis=1)
# tests predict_to_user_average in the class RecSys
rs = RecSys(data)
yp = rs.predict_to_user_average(csr_rownnz, intrsct.getnnz(axis=1))
print(rs.rmse(yp))
unions = nnz_i + csr_rownnz[intrsct.indices] - intrsct.data
Hidden Tests Redacted
        return intrsct.data / unions

```

```

        out = csr_matrix((sim, intrsct.indices, intrsct.indptr), intrsct.shape)

```

1b. Complete the function predict_to_user_average in the class RecSys [10 pts]

```

        return pairwise_jaccard_sparse(Xr.T)
        #from scipy.spatial import distance
Hint: Include rated items only when averaging
        #x = Xr.I
        #return 1-distance.cdist(x, x, 'jaccard') #1/(1+distance.cdist(self.Mm,
In [6]: self.Mm, 'jaccard'))
        #from sklearn.decomposition import PCA

```

```

Grade cell: cell-dbe3ae170d198f001 Score: 15.38 / 15.38 (Top)
        #x = PCA(n_components=100).fit_transform(x)
        #return 1-distance.cdist(x, x, 'jaccard') #1/(1+distance.cdist(self.Mm,
self.Mm, 'jaccard'))
# tests predict_to_user_average in the class RecSys
#return 1-distance.cdist(Xr.I, 'jaccard').reshape(Xr.shape[0],Xr.shape
yp = rs.predict_to_user_average()
#distance.cdist(Xr.I, Xr.I, 'jaccard') #1/(1+distance.cdist(self.Mm, self.
print(rs.rmse(yp))
Mm, 'jaccard')
Hidden Tests Redacted

```

Q2. Content-Based model [25 pts]

2a. Complete the function calc_movie_feature_matrix in the class ContentBased [5 pts]

In [7]:

```
cb = ContentBased(data)
```

In [8]:

```

Grade cell: cell-232d6022e1da2f62 Score: 0.0 / 7.69 (Top)
# tests calc_movie_feature_matrix in the class ContentBased
assert(cb.Mm.shape==(3883, 18))
Hidden Tests Redacted

```

```

-----
-----
KeyError                                Traceback (most recent call
last)
/opt/conda/lib/python3.7/site-packages/pandas/core/indexes/base.py i
n get_loc(self, key, method, tolerance)
Traceback Redacted

```

KeyError: 440

2b. Complete the function `calc_item_item_similarity` in the class `ContentBased` [10 pts]

This function updates `self.sim` and does not return a value.

Some factors to think about:

1. The movie feature matrix has binary elements. Which similarity metric should be used?
2. What is the computation complexity (time complexity) on similarity calculation?
Hint: You may use functions in the `scipy.spatial.distance` module on the dense matrix, but it is quite slow (think about the time complexity). If you want to speed up, you may try using functions in the `scipy.sparse` module.

In [9]:

```
cb.calc_item_item_similarity()
```

In [10]:

Grade cell: cell-b1260c5ba472d43b

Score: 3.08 / 3.08 (Top)

```
# tests calc_item_item_similarity in ContentBased class  
Hidden Tests Redacted
```

In [11]:

Grade cell: cell-4a8dc462fa8d33f0

Score: 4.62 / 4.62 (Top)

```
# additional tests for calc_item_item_similarity in ContentBased class  
Hidden Tests Redacted
```

In [12]:

Grade cell: cell-3b1eb8e79527f012

Score: 1.54 / 1.54 (Top)

```
# additional tests for calc_item_item_similarity in ContentBased class  
Hidden Tests Redacted
```

In [13]:

Grade cell: cell-64689afe942eba5a

Score: 1.54 / 1.54 (Top)

```
# additional tests for calc_item_item_similarity in ContentBased class  
Hidden Tests Redacted
```

In [14]:

Grade cell: cell-a7be29cab0d2e38a

Score: 4.62 / 4.62 (Top)

additional tests for calc_item_item_similarity in ContentBased class
Hidden Tests Redacted

2c. Complete the function predict_from_sim in the class RecSys [5 pts]

In [15]:

Grade cell: cell-a83193bd93c45044

Score: 7.69 / 7.69 (Top)

tests for predict_from_sim in RecSys class
Hidden Tests Redacted

2d. Complete the function predict in the class RecSys [5 pts]

After completing the predict method in the RecSys class, run the cell below to calculate rating prediction and RMSE. How much does the performance increase compared to the baseline results from above?

In [16]:

```
yp = cb.predict()
rmse = cb.rmse(yp)
print(rmse)
```

1.0128116783754684

In [17]:

Grade cell: cell-a8a3fa60da47abcf

Score: 7.69 / 7.69 (Top)

tests method predict in the RecSys class
Hidden Tests Redacted

Q3. Collaborative Filtering

3a. Complete the function cossim in the class Collaborative [10 pts]

To Do:

1. Impute the unrated entries in self.Mr to the user's average rating then subtract by the user mean, call this matrix X.

2. Calculate cosine similarity for all item-item pairs. Don't forget to rescale the cosine similarity to be 0~1.

You might encounter divide by zero warning (numpy will fill nan value for that entry). In that case, you can fill those with appropriate values.

Hint: Let's say a movie item has not been rated by anyone. When you calculate similarity of this vector to another, you will get $\vec{0}=[0,0,0,\dots,0]$. When you normalize this vector, you'll get divide by zero warning and it will make nan value in self.sim matrix. Theoretically what should the similarity value for $\vec{x}_i \cdot \vec{x}_i$ when $\vec{x}_i = \vec{0}$? What about $\vec{x}_i \cdot \vec{x}_j$ when $\vec{x}_i = \vec{0}$ and $\vec{x}_j$ is an any vector?

Hint: You may use `scipy.spatial.distance.cosine`, but it will be slow because its cosine function does vector-vector operation whereas you can implement matrix-matrix operation using numpy to calculate all cosines all at once (it can be 100 times faster than vector-vector operation in our data). Also pay attention to the definition. The `scipy.spatial.distance` provides distance, not similarity.

1. Run the below cell that calculate yp and RMSE.

In [18]:

```
cf = Collaborative(data)
cf.calc_item_item_similarity(cf.cossim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
```

1.0263081279662305

In [19]:

Grade cell: cell-126fb0d12b2c66e7

Score: 1.54 / 1.54 (Top)

```
# tests cossim method in the Collaborative class
Hidden Tests Redacted
```

In [20]:

Grade cell: cell-9e6bc1c31e566d76

Score: 3.08 / 3.08 (Top)

additional tests for cossim method in the Collaborative class
Hidden Tests Redacted

In [21]:

Grade cell: cell-7b9c0acd25eaadda

Score: 1.54 / 1.54 (Top)

additional tests for cossim method in the Collaborative class
Hidden Tests Redacted

In [22]:

Grade cell: cell-369189aa9a403a0f

Score: 3.08 / 3.08 (Top)

additional tests for cossim method in the Collaborative class
Hidden Tests Redacted

In [23]:

Grade cell: cell-bfde47a18fccd03d

Score: 3.08 / 3.08 (Top)

additional tests for cossim method in the Collaborative class
Hidden Tests Redacted

In [24]:

Grade cell: cell-cb7766e929ce12ef

Score: 3.08 / 3.08 (Top)

additional tests for cossim method in the Collaborative class
Hidden Tests Redacted

3b. Complete the function jacsim in the class Collaborative [15 pts]

3b [15 pts] = 3b-i) [5 pts]+3b-ii) [5 pts]+ 3b-iii) [5 pts]

Function `jacsim` calculates jaccard similarity between items using collaborative filtering method. When we have a rating matrix `self.Mr`, the entries of `Mr` matrix are 0 to 5 (0: unrated, 1-5: rating). We are interested to see which threshold method works better when we use jaccard similarity in the collaborative filtering.

We may treat any rating 3 or above to be 1 and the negatively rated (below 3) and no-rating as 0. Or, we may treat movies with any ratings to be 1 and ones that has no rating as 0. In this question, we will complete a function `jacsim` that takes a transformed rating matrix `X` and calculate and returns a jaccard similarity matrix.

Let's consider these input cases for the utility matrix M_r , with ratings 1-5 and 0s for no-rating.

1. $M_r \geq 3$
2. $M_r \geq 0$
3. M_r , no transform.

Things to think about:

- The cases 1 and 2 are straightforward to calculate Jaccard, but what does Jaccard mean for multicategory data?
- Time complexity: The matrix M_r is much bigger than the item feature matrix M_m , therefore it will take very long time if we calculate on dense matrix.
Hint: Use sparse matrix.
- Which method will give the best performance?

3b-i) When $M_r \geq 3$ [5 pts]

After you've implemented the `jacsim` function, run the code below. If implemented correctly, you'll have RMSE below 0.99.

In [25]:

```
cf = Collaborative(data)
Xr = cf.Mr>=3
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jacsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<0.99)
```

similarity calculation time 0.8725570384413004

0.9819058692126349

In [26]:

Grade cell: cell-a80d917298a5c254

Score: 3.08 / 3.08 (Top)

tests RMSE for jacsim implementation
Hidden Tests Redacted

In [27]:

Grade cell: cell-fdebe2b922817f71

Score: 1.54 / 1.54 (Top)

additional tests for RMSE for jacsim implementation
Hidden Tests Redacted

In [28]:

Grade cell: cell-ba2fba2da48d3160

Score: 1.54 / 1.54 (Top)

additional tests for jacsim implementation
Hidden Tests Redacted

In [29]:

Grade cell: cell-f7e1c6f239e3155f

Score: 1.54 / 1.54 (Top)

additional tests for jacsim implementation
Hidden Tests Redacted

3b-ii) When $M_r \geq 1$ [5 pts]

After you've implemented the jacsim function, run the code below. If implemented correctly, you'll have RMSE below 1.0.

In [30]:

```
cf = Collaborative(data)
Xr = cf.Mr>=1
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jacsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<1.0)
```

similarity calculation time 1.0741165820509195

0.991363571262366

In [31]:

Grade cell: cell-6103dd685a33c891

Score: 3.08 / 3.08 (Top)

tests RMSE for jacsim implementation
Hidden Tests Redacted

In [32]:

Grade cell: cell-905be3dcffdd989b

Score: 1.54 / 1.54 (Top)

tests RMSE for jacsim implementation
Hidden Tests Redacted

In [33]:

Grade cell: cell-53a7044088d56bae

Score: 1.54 / 1.54 (Top)

tests jacsim implementation
Hidden Tests Redacted

In [34]:

Grade cell: cell-113417d673195815

Score: 1.54 / 1.54 (Top)

tests performance of jacsim implementation
Hidden Tests Redacted

3b-iii) When M_r ; no transform [5 pts]

After you've implemented the jacsim function, run the code below. If implemented correctly, you'll have RMSE below 0.96

In [35]:

```
cf = Collaborative(data)
Xr = cf.Mr.astype(int)
t0=time.perf_counter()
cf.calc_item_item_similarity(cf.jacsim,Xr)
t1=time.perf_counter()
time_sim = t1-t0
print('similarity calculation time',time_sim)
yp = cf.predict()
rmse = cf.rmse(yp)
print(rmse)
assert(rmse<0.96)
```

similarity calculation time 183.650583518669

0.9509126236828654

In [36]:

Grade cell: cell-576df690bad55235

Score: 3.08 / 3.08 (Top)

tests jacsim implementation RMSE
Hidden Tests Redacted

In [37]:

Grade cell: cell-270a003313f847ff

Score: 1.54 / 1.54 (Top)

tests jacsim implementation RMSE
Hidden Tests Redacted

In [38]:

Grade cell: cell-a5efeaa7f4bf26a9

Score: 1.54 / 1.54 (Top)

tests jacsim implementation
Hidden Tests Redacted

In [39]:

Grade cell: cell-15304b41c7ed15d3

Score: 0.0 / 1.54 (Top)

tests jaccsim implementation performance
Hidden Tests Redacted

AssertionError Traceback (most recent call
1 last)
<ipython-input-39-8d5d5375f65d> in <module>
Traceback Redacted

AssertionError: Check jaccsim implementation. Implementation is taking too long to run.

3.C Discussion [Peer Review]

Answer the questions below in this week's Peer Review assignment.

1. Summarize the methods and performances: Below is a template/example.

Method	RMSE
Baseline, $Y_p=3$	
Baseline, $Y_p = \mu_u$	
Content based, item-item	
Collaborative, cosine	
Collaborative, jaccard, $M_r \geq 3$	
Collaborative, jaccard, $M_r \geq 1$	
Collaborative, jaccard, M_r	

1. Discuss which method(s) work better than others and why.

In []: